CyberNexus

AI-Powered Cybersecurity Learning Platform

Full-Stack Cybersecurity Education • Hands-On Labs • CTF • AI • Real-Time Security

Final Soutenance Report — 2026 Edition

Author: Yassine Kalthoum , software engineering master's student at Woolf University | **Orientation:** Software , Network Engineering • Cybersecurity

Deployment: Google Cloud Platform — Google Cloud Run | **Database:** MongoDB Atlas • Redis | **AI:** Google Gemini / Google GenAI SDK

Executive Summary & Problem Statement

CyberNexus is a full-stack cybersecurity learning platform and interactive cyber-range concept designed to **bridge the gap between theoretical cybersecurity education and practical technical execution**. The application combines structured lessons, interactive diagrams, controlled laboratories, CTF-style challenges, a simulated Kali-style terminal, an AI cybersecurity tutor, real-time communication, role-based administration, instructor telemetry, gamification, certificates, payment-advance course access, and bilingual user experience.

The project is implemented as a JavaScript-based **React/Vite frontend** and a **Node.js/Express backend**. MongoDB/Mongoose provides persistent application data, Redis provides fast temporary data and infrastructure support, Socket.IO enables real-time communication, and Google Gemini is integrated through the server side for the Nexus AI assistant. The production application is deployed on Google Cloud Platform using Google Cloud Run as the main hosting environment.

Problem: Traditional cybersecurity learning separates theory from execution. Learners may understand definitions but lack a controlled environment in which to practice reconnaissance, web security, defensive analysis, command-line concepts, and security workflows. Instructors need visibility into learner progress; administrators need secure control over identities, roles, and platform operations.

Solution: CyberNexus centralizes the learning lifecycle into one platform — connecting educational content with controlled practical activities, challenges, AI-assisted explanations, progress measurement, and administrative supervision.

Learn

Structured cybersecurity lessons and progressive learning paths.

Practice

Transform concepts into controlled labs and simulations.

Challenge

Reinforce knowledge using quizzes, CTF-style tasks, XP, achievements, and streaks.

Assist

Context-aware AI cybersecurity tutor through Google Gemini.

Measure

Track progress, activity, submissions, certificates, and instructor/admin metrics.

Secure

Defense-in-depth controls across web, API, authentication, data, and administration layers.

Deploy

Cloud-native GCP deployment via Google Cloud Run.

Educational Philosophy: Learn → Practice → Challenge → Defend → Measure → Progress

Development Methodology

The project follows an **iterative full-stack engineering workflow**. Development is organized around progressive implementation of the platform's core layers: user interface, backend APIs, persistence, security, real-time services, learning logic, AI integration, and cloud deployment.

Phase 1 — Analysis

1

Identify learners, instructors, administrators, learning scenarios, and security requirements. **Expected result:** Functional and security requirements.

Phase 2 — Architecture

2

Separate client presentation, server/API, database, cache, and real-time services architecture. **Expected result:** Maintainable layered architecture.

Phase 3 — Frontend

3

Build React/Vite pages, dashboards, labs, roadmap, terminal interface, and interactivity. **Expected result:** Interactive learning experience.

Phase 4 — Backend

4

Implement Express APIs, models, authentication, RBAC, learning logic, and controllers. **Expected result:** Secure APIs and controllers.

Phase 5 — Security

5

Add Helmet, CORS validation, rate limiting, sanitization, CSRF protection, and access controls. **Expected result:** Enforced security and access controls.

Phase 6 — Integration

6

Connect MongoDB Atlas, Redis, Socket.IO, Google Gemini, and Stripe API for payments. **Expected result:** Integrated platform payments.

Phase 7 — Validation

7

Test local flows, database connectivity, real-time communication, and production features. **Expected result:** Verified functionality and production features.

Phase 8 — Deployment

8

Containerize and deploy to Google Cloud Platform / Cloud Run. **Expected result:** Cloud-hosted application.

System Architecture & Technology Stack

Decoupled Full-Stack Architecture

CyberNexus uses a decoupled architecture where the **React 19 + Vite + Tailwind CSS** client handles presentation and learner interaction, while the **Node.js + Express** server centralizes business logic, security, authentication, data access, and external service integration.

The backend exposes health and security-oriented status endpoints, while the **Socket.IO** layer supports live activity and communication. External integrations include **Google OAuth**, **EmailJS**, and a **payment flow**. The entire system is hosted on **Google Cloud Platform** via **Google Cloud Run**.

Core engineering principle: **security is treated as a cross-cutting concern**. The application uses layered controls including Helmet security headers, strict CORS origin validation, rate limiting, CSRF protection, XSS sanitization, MongoDB/NoSQL sanitization, HTTP Parameter Pollution protection, request validation, secure cookies, authentication, authorization, centralized logging, and security status endpoints.

Technology Stack

Frontend	React 19 + Vite 6 + Tailwind CSS + Motion + Lucide React
Backend	Node.js + Express 4
Database	MongoDB Atlas + Mongoose
Cache	Redis
Realtime	Socket.IO
Auth	JWT + Passport + Google OAuth
Security	Helmet, CORS, rate-limit, mongo-sanitize, xss-clean, hpp
AI	@google/genai / Gemini
Certificates	jsPDF + QRCode
Email	EmailJS
Cloud	Google Cloud Platform / Cloud Run
Containerization	Docker / Docker Compose

Learning Experience & Cybersecurity Curriculum

The learning model is **progressive**. Learners move from foundations to specialized security domains while receiving immediate feedback through quizzes, labs, CTFs, and progression metrics.

Learning Components

Paths

Organize courses and learning journeys.

Lessons

Provide structured concepts, objectives and material.

Quizzes

Validate comprehension and award XP.

Labs

Convert theory into controlled technical practice.

CTFs

Challenge-based security problem solving.

Roadmap

Visualize progression from beginner to expert.

Tools

Industry-standard security utilities.

Terminal

Command-oriented workflows in a simulator.

Progress

Learning completion and learner metrics.

Roadmap Stations

1. **Station 01** — IT Foundations & Networking
2. **Station 02** — Reconnaissance & Network Scanning
3. **Station 03** — Web Application Security & OWASP Top 10
4. **Station 04** — System Exploitation & Active Directory
5. **Station 05** — SOC Defense, SIEM & Threat Hunting
6. **Station 06** — Reverse Engineering & Malware Analysis
7. **Station 07** — Cloud Security, Zero Trust & CISO Elite

Career Tracks

All-Rounder, Offensive Red Team, Defensive Blue Team

Toolkit

Nmap, Wireshark, Burp Suite, Metasploit Framework, John the Ripper, OWASP ZAP, Ghidra, THC-Hydra, SQLmap, and Aircrack-ng

Hands-On Labs & CTFs

Practical topics include SQL Injection, Cross-Site Scripting, NoSQL Injection, CSRF/session defense, web application security, defensive concepts, telemetry, and incident-oriented exercises. The CTF subsystem uses difficulty tiers: **200 XP** (Easy), **400 XP** (Medium), **600 XP** (Hard), **800 XP** (Insane). The browser terminal is an **educational simulator** — not a real privileged Kali Linux environment. All practical exercises are intended for systems owned by the learner or explicitly authorized for testing.

Nexus AI, Authentication & RBAC

Nexus AI — Google Gemini Integration

Nexus AI is the platform's cybersecurity learning assistant. The frontend communicates with a backend chat endpoint (`POST /api/chat`), while the server delegates AI requests to a dedicated Google GenAI/Gemini service layer. This keeps the AI credential **server-side** instead of exposing it in browser code.

The educational design emphasizes **guidance rather than simply revealing challenge answers**. The assistant can explain concepts, security tooling, attack/defense flows, and learning content while remaining part of the controlled training environment.

Authentication & RBAC

Three primary roles are defined: **Student**, **Instructor**, and **Admin**. The application dynamically exposes privileged navigation and APIs according to authenticated permissions.

Capability	Student	Instructor	Admin
Courses / Lessons	✓	✓	✓
Labs / CTF	✓	✓	✓
Nexus AI	✓	✓	✓
Instructor telemetry	—	✓	✓
Grade / inspect submissions	—	✓	✓
Admin panel	—	—	✓
Change user roles	—	—	✓
Modify XP / level / streak	—	—	✓
Ban / restore accounts	—	—	✓
Delete users	—	—	✓
Security audit logs	—	—	✓

Privilege escalation protection: Students and instructors cannot promote themselves to privileged roles. Role assignment is restricted to authorized administrators. Sessions use **HTTP-only cookies with SameSite policies**, with Bearer-token fallback where embedded preview contexts require it. Logout clears session state and redirects the user to a clean home state.

Security Architecture — Defense in Depth

Security is embedded at **multiple layers** rather than relying on a single mechanism. The application uses layered controls including Helmet security headers, strict CORS origin validation, rate limiting, CSRF protection, XSS sanitization, MongoDB/NoSQL sanitization, HTTP Parameter Pollution protection, request validation, secure cookies, authentication, authorization, centralized logging, and security status endpoints.

Layer	Control	Security Objective
HTTP	Helmet + CSP + security headers	Reduce browser-side attack surface.
Transport/API	Strict CORS origin validation	Prevent unauthorized cross-origin requests.
Availability	Rate limiting	Reduce brute-force and request-flooding risk.
Input	mongo-sanitize + NoSQL guards	Reduce MongoDB operator injection.
Input	xss-clean	Sanitize XSS-oriented payloads.
Input	hpp	Mitigate HTTP Parameter Pollution.
Validation	express-validator	Validate request data and constraints.
Session	HTTP-only cookies + SameSite	Reduce token exposure to client scripts.
Authentication	JWT / Passport / OAuth / bcrypt	Secure identity lifecycle.
Authorization	RBAC middleware	Enforce least privilege.
CSRF	CSRF middleware + token endpoint	Protect state-changing requests.
Logging	Request and security logs	Provide observability and auditability.
API responses	No-store cache policy	Reduce sensitive response persistence.
Platform	GCP / Cloud Run	Managed cloud deployment and scaling.

Configured browser policies include: Content-Security-Policy directives, frame-ancestor restrictions, object blocking, referrer policy, cross-origin policies, Permissions-Policy, and sensitive API cache prevention. CORS uses an **allow-list approach** and explicitly recognizes configured origins, localhost, Cloud Run domains, and AI Studio preview domains. Unauthorized origins are rejected. Rate limiting: API requests are limited to **200 requests per IP per 15-minute window** in the current server configuration.

Real-Time Communication, Gamification & Certificates

Real-Time Communication

Socket.IO is attached to the Node.js HTTP server. The real-time layer supports **live chat, security feeds, active-user information, notifications, and activity events**. The same CORS policy is applied to Socket.IO connections. The administrative interface provides visibility into platform activity and security logs, helping instructors and administrators understand system state rather than relying only on static page refreshes.

Gamification & Progression Model

CyberNexus uses XP, levels, and streaks to turn learning activity into measurable progression.

XP required for next level = Current Level × 1,000

Cumulative XP for Level L = ((L − 1) × L / 2) × 1,000

Level	Rank	XP to next level	Cumulative XP
1	Novice / Script Kiddie	1,000	0
2	Apprentice	2,000	1,000
3	Junior Operator	3,000	3,000
4	Cyber Practitioner	4,000	6,000
5	Security Analyst	5,000	10,000
6	Penetration Tester	6,000	15,000
7	Senior Specialist	7,000	21,000
8	Threat Hunter	8,000	28,000
9	Cyber Architect	9,000	36,000
10+	Elite	9,000+	45,000+

XP sources: lessons/quizzes award **150–350 XP**, lab subtasks **50–150 XP**, full lab completion **300–600 XP**, and CTFs **200/400/600/800 XP** for Easy/Medium/Hard/Insane. Streak multipliers encourage consistent practice.

Certificates, Payments & Internationalization

Certificates: The frontend contains a PDF certificate generator using **jsPDF** and QR-code support. The backend exposes a verification endpoint and a dedicated verification service. This creates a learner-facing achievement workflow rather than a static completion screen.

Payments: The platform includes a payment-aware course access flow. Screenshots in the repository demonstrate locked paid courses for students and privileged access for administrators, as well as a crypto-payment/unlocking interface.

Internationalization: English and French are supported through a translation layer. The user can switch language and persist preferences.

UX: The platform includes dark/light themes, responsive dashboards, interactive animations, iconography, notifications, and persistent user preferences.

REST API, Data Architecture & GCP Deployment

REST API Specification

Area	Endpoint	Purpose
Auth	POST /api/auth/register	Register student account.
Auth	POST /api/auth/login	Authenticate user.
Auth	POST /api/auth/logout	Terminate session.
Auth	GET /api/auth/me	Return current profile/metrics.
Admin	GET /api/admin/overview	Platform health and metrics.
Admin	GET /api/admin/users	Retrieve user roster.
Admin	PUT /api/admin/users/:id/role	Change role.
Admin	PUT /api/admin/users/:id/stats	Adjust XP/level/streak.
Admin	POST /api/admin/users/:id/toggle-ban	Ban/restore user.
Admin	DELETE /api/admin/users/:id	Delete user.
Admin	GET /api/admin/logs	Audit/security logs.
Admin	POST /api/admin/logs/clear	Clear log history.
Instructor	GET /api/instructor/stats	Class metrics.
Instructor	GET /api/instructor/students	Student roster/progress.
Instructor	GET /api/instructor/submissions	Inspect lab submissions.
AI	POST /api/chat	Send contextual AI query.
Verification	GET /api/verify-certificate/:id	Validate certificate.

Infrastructure/status endpoints include database status, Redis status, security status, and CSRF-token retrieval. The API layer is mounted beneath the Express application and protected by the common middleware stack.

Data & Persistence Architecture

MongoDB/Mongoose provides the main persistent store. The repository contains models for users, profiles, lessons, quizzes, labs, and certificates. Security and learning state are represented as application data rather than only browser state. **Redis** provides cache, temporary state, and real-time infrastructure support. The project includes database auto-seeding utilities for initial data population.

Google Cloud Platform Deployment

The production environment is deployed on **Google Cloud Platform**. **Google Cloud Run** is used as the primary application hosting service — the project is designed to run as a managed cloud service, not only a local prototype.

Deployment advantages: managed HTTPS, container-based delivery, automatic scaling, revision management, centralized logs, and integration with other Google Cloud services. The server configuration accounts for reverse-proxy operation and Cloud Run origins, including trust-proxy behavior and allowed `*.run.app` domains.

Recommended production secret management: use Cloud Run environment variables and, preferably, Google Cloud Secret Manager for sensitive credentials such as JWT secrets, session secrets, OAuth secrets, and Gemini API keys.

Containerization & Build Workflow

The project includes Docker-related configuration and a `docker-compose.yml` file. Containerization improves reproducibility between development and deployment.

```
npm install
npm run build
npm start

# Container-oriented workflow
docker build -t cybernexus .
docker run -p 3000:3000 cybernexus

# Cloud build/deployment
gcloud builds submit --tag <container-image>
gcloud run deploy <service> --image <container-image>
```

Key Environment Variables

```
NODE_ENV=production
PORT=8080
MONGO_URI=<MongoDB Atlas connection string>
JWT_SECRET=<strong secret>
SESSION_SECRET=<strong secret>
REDIS_URL=<Redis connection string>
GOOGLE_CLIENT_ID=<OAuth client ID>
GOOGLE_CLIENT_SECRET=<OAuth client secret>
GEMINI_API_KEY=<Gemini key>
CLIENT_URL=<production Cloud Run URL>
ALLOWED_ORIGINS=<trusted origins>
```

Security rule: Secrets must never be committed to Git. Production credentials should be rotated if accidentally exposed and stored through a secure secret-management mechanism.

Results, Future Roadmap & Conclusion

Results & Achievements

Area	Result Demonstrated
Full-stack engineering	React/Vite frontend + Node/Express backend.
Cybersecurity education	Lessons, roadmap, labs, CTFs, toolkit, and terminal simulator.
Artificial intelligence	Server-side Google Gemini integration through Nexus AI.
Authentication	JWT/session/OAuth-oriented authentication stack.
Authorization	Student/Instructor/Admin RBAC.
Security engineering	Defense-in-depth middleware and browser policies.
Persistence	MongoDB Atlas/Mongoose models and seeding.
Performance infrastructure	Redis service integration.
Realtime	Socket.IO communication and security feeds.
Certification	PDF generation + verification endpoint.
Internationalization	English/French support.
Business model	Paid course locking/unlocking workflow.
Cloud	Google Cloud Platform / Cloud Run deployment.
Documentation	README + full technical guide + evidence screenshots.

Current Limitations

- The Kali-style terminal is a simulator; not a privileged remote Kali environment.
- Practical labs should be expanded with isolated, containerized targets for deeper cyber-range realism.
- Production secrets should be managed with a dedicated secret manager.
- Automated end-to-end testing could be expanded beyond the current validation/evidence workflow.
- A formal CI/CD pipeline with automated security scanning would further strengthen production operations.
- Advanced observability could integrate centralized metrics, tracing, and alerting.
- The payment flow should be connected to a fully production-grade payment provider and webhook verification if monetization is enabled.
- The AI assistant should continue to use strong server-side controls, quotas, and prompt/response safety policies.

Future Roadmap

- Advanced SOC simulation and SIEM training.
- Isolated Docker-based vulnerable environments for labs.
- Automated lab grading and richer submission analytics.
- Team-based CTF competitions and leaderboards.
- Cloud security modules for AWS, Azure, and GCP.
- Container and Kubernetes security laboratories.
- Digital forensics and malware-analysis environments.
- Threat intelligence and detection-engineering exercises.
- More advanced AI tutoring and personalized learning recommendations.
- Automated CI/CD security testing, SAST, dependency scanning, and container scanning.
- Centralized monitoring, tracing, and alerting.
- Expanded certificate verification and institutional reporting.

Conclusion: CyberNexus demonstrates how software engineering, cybersecurity, artificial intelligence, real-time systems, database engineering, and cloud deployment can be combined into one coherent educational platform. The project addresses a real learning problem: cybersecurity requires practical repetition, controlled experimentation, and measurable progression. CyberNexus responds with an integrated environment that connects theory, labs, challenges, AI guidance, real-time communication, instructor supervision, and administrative security.

From an engineering perspective, the project demonstrates modular architecture, REST APIs, MongoDB persistence, Redis integration, Socket.IO communication, authentication and RBAC, security middleware, certificate generation, third-party integrations, containerization, and Google Cloud Platform deployment.

Final message for the soutenance: CyberNexus is not only a learning website. It is an extensible cybersecurity training ecosystem whose architecture is designed to evolve toward a more realistic cloud-hosted cyber range.

Acknowledgements: GOMYCODE — for the practical learning environment and project-oriented development experience. Woolf University — for the academic/software-engineering foundation supporting the project. Instructors and mentors — for guidance, feedback, and technical direction throughout the project. Open-source community — for the frameworks, libraries, and security tooling that make modern software development possible.

LEARN PRACTICE CHALLENGE DEFEND MEASURE PROGRESS